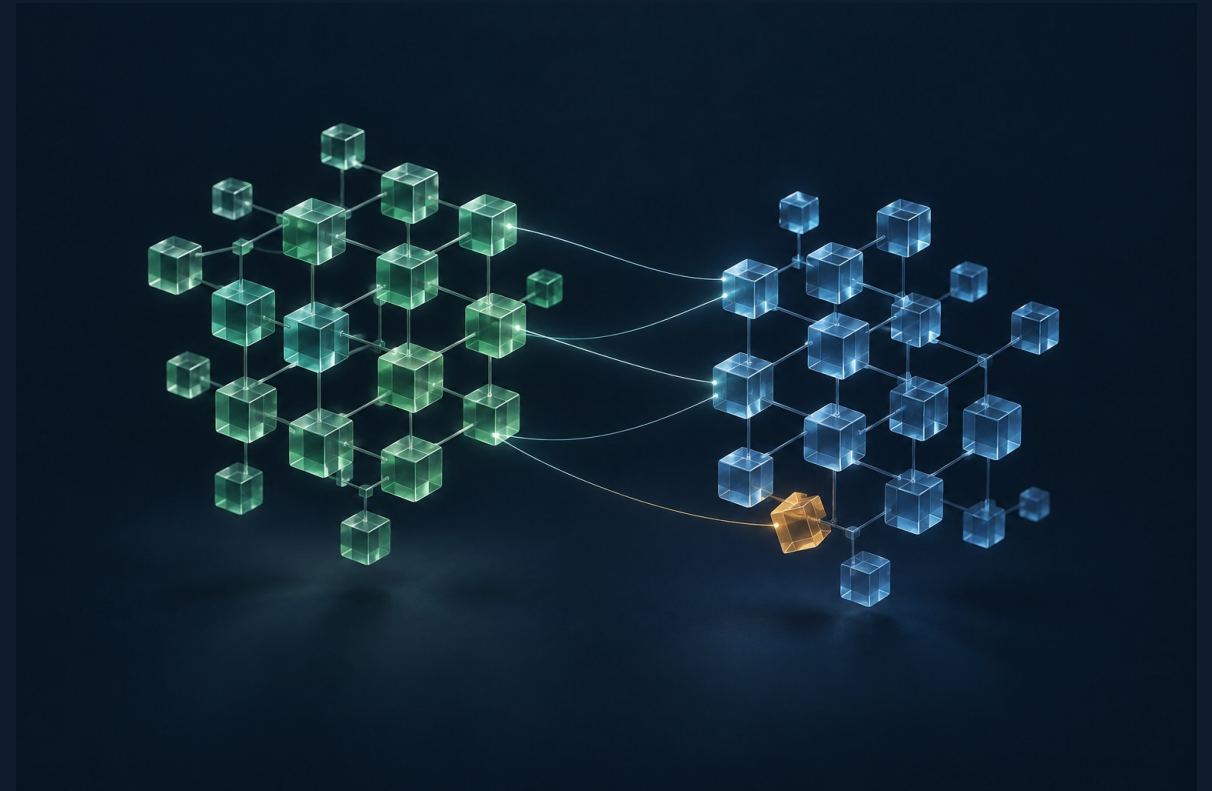


SAGE

Project update

What is done
and what is pending

Updated October 6, 2026



The project in one slide

**Scientific software in Rust and C++20
with checks against the original implementation.**

SAGE runs comparisons and saves the evidence.

Einsums is the current library under evaluation.

Both ports cover the CPU library and Python interface.

GPU support is outside this scope.

What is already done

Rust translation

90 CPU Python exports and 16 helpers.
Installable package backed by Rust.

C++20 translation

90 CPU Python exports and 16 helpers.
Independent native C++ backend.

SymSan comparisons

C++20 passed 128 new cases across 23 operations
against Einsums and NumPy.

Formal proofs

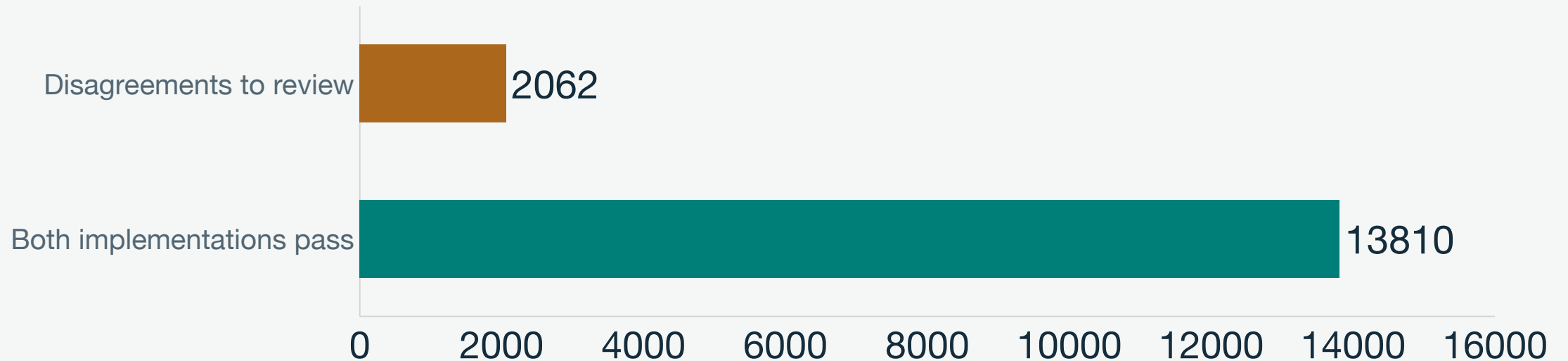
Six bounded operations proved in both languages.
Eligible public calls use those methods.

Both ports implement the CPU/Python surface. Exact library compatibility remains pending.

Saved Rust comparison results

128 generated inputs

15,872 comparison checks



Rust met every numerical check in this replay.

Full-library compatibility remains incomplete.

C++20: 128/128 fresh cases pass. API replay: 15,872 checks with 2,056 disagreements.

What is still pending

Compatibility review

Review 2,062 Rust and 2,056 C++ replay disagreements against agreed contracts.

Proof coverage

Extend proofs to larger sizes, other dtypes and upstream BLAS.

Remaining behaviors

Vary numerical inputs, explore computation paths and automate further translation.

Real application use

Check Psi4 integration and measure performance against the original library.

Completed checks and proof limits

Implementation checks

129 project tests passed.

C++20 passed 4,416 native cases and sanitizer checks.

Six operations proved and connected

Copy, add, multiply, transpose, matmul and scale.

144 proof obligations per language. Eligible public calls use that code.

Bounds: float64, dimensions 1–4, signed-byte values / 8.

Proposed next milestones

- | | |
|---------------------------------|--|
| 1 Shared contracts | Classify disagreements and agree on behavior for both ports. |
| 2 Calling contracts | Check complete calling behavior, C++ overloads and installation contracts. |
| 3 Broader verification | Prove adapters. Extend shapes and dtypes. Verify upstream BLAS contracts. |
| 4 Application validation | Run downstream regressions and benchmark native performance. |

Immediate work: compatibility review and broader verification of both ports.